Must Love Scrubs — Database Schema & API Contract (v1 draft)

Companion to the Master Specification. Turns §5 (data model) and §6 (question engine) into concrete tables and endpoints. Tech-neutral but opinionated: PostgreSQL (relational core + JSONB for template bodies + an `edges` table for the knowledge graph), REST/JSON API, JWT auth. Same API serves the web app now and the native app later.

Status: draft for approval. No app is built yet.

---

0. Conventions

- **IDs:** content uses readable keys `TYPE-SYS-####` (e.g. `DIS-CV-0001`). Users/attempts/posts use UUIDs.
- **Timestamps:** UTC ISO-8601; every table has `created_at`, `updated_at`.
- **Bodies:** structured content stored as **JSONB** validated against a template (see §3). Tags/relationships are real columns/rows so they're queryable.
- **Auth:** JWT bearer. `Authorization: Bearer <token>`. Roles: `visitor`, `member`, `subscriber`, `author`, `admin`.
- **Errors:** `{ "error": { "code": "string", "message": "human text", "details": {} } }` with standard HTTP status.
- **Pagination:** cursor-based — `?limit=25&cursor=...` → `{ "data": [...], "next_cursor": "..." }`.
- **Gating:** every content/question row has a `free` boolean; the API filters by the caller's entitlement. Gating is data, never code forks.

---

1. Enumerations

```
content_type : NKI DIS MED LAB PRO SKL ANA SYM ASM QBK NGN FLS VID IMG EDU PATH
body_system  : cardiovascular respiratory neurological renal endocrine
gastrointestinal
               hematologic immune_infectious musculoskeletal integumentary
reproductive_ob
               neonatal pediatric psychiatric
client_need  : SMS HPM PSY BPI PHA RRT PAD          (NCLEX Client Needs)
cj_step      : REC ANA PRI GEN ACT EVA              (NGN clinical judgment)
difficulty   : 1 2 3 4 5                            (1 foundational … 5 complex
NGN)
exam         : RN PN SPECIALTY
age_group    : adult pediatric neonatal geriatric
```

```
care_setting : er icu med_surg ld pediatrics community ltc
status       : draft in_review published archived
question_type: mc sata matrix bowtie dropdown ordered hotspot trend audio image
casestudy
relation     : has_symptom treated_by monitored_with causes complication_of
tested_by
               teaches assesses part_of links_to precedes           (typed
edges)
post_kind    : post reply
```

---

2. Tables (illustrative DDL)

2.1 `nodes` — every content object

```
nodes (
  id            text PRIMARY KEY,           -- DIS-CV-0001
  type          content_type NOT NULL,
  title         text NOT NULL,
  slug          text UNIQUE NOT NULL,
  body_system   body_system,
  specialties   text[] DEFAULT '{}',        -- 22 any-RN specialties
  difficulty    smallint,                   -- 1..5
  client_needs  client_need[] DEFAULT '{}',
  cj_steps      cj_step[] DEFAULT '{}',
  age_groups    text[] DEFAULT '{}',
  care_settings text[] DEFAULT '{}',
  exam          exam DEFAULT 'RN',
  free          boolean DEFAULT false,
  status        status DEFAULT 'draft',
  version       int DEFAULT 1,
  body          jsonb NOT NULL DEFAULT '{}',-- template-shaped content (§3)
  search_terms  text[] DEFAULT '{}',        -- synonyms, abbreviations,
brand/generic
  search_tsv    tsvector,                    -- full-text index
  author_id     uuid, reviewer_id uuid, reviewed_at timestamptz,
  created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()
)
-- indexes: type, body_system, difficulty, status, GIN(search_tsv),
--          GIN(specialties), GIN(client_needs)
```

2.2 `edges` — the knowledge graph

```
edges (
  from_id  text REFERENCES nodes(id) ON DELETE CASCADE,
  to_id    text REFERENCES nodes(id) ON DELETE CASCADE,
```

```
  relation relation NOT NULL,
  weight   real DEFAULT 1.0,             -- strength; drives adaptive priority
  PRIMARY KEY (from_id, to_id, relation)
)
-- index: (from_id), (to_id)
```

2.3 `questions` — extends a node of type QBK/NGN

Question payload lives in `nodes.body` (JSONB, shape in §4) so it inherits all tags/edges. A thin view exposes them:

```
-- questions are nodes WHERE type IN ('QBK','NGN')
-- body JSONB holds: qtype, stem, opts[], correct, tip, pearl, references[],
--                   learning_objective, bloom, est_time_sec  (see §4)
```

2.4 People & entitlements

```
users (
  id uuid PK, email citext UNIQUE, name text, role text DEFAULT 'member',
  exam_date date, target text,            -- e.g. pass-target
  created_at, updated_at )

entitlements (
  user_id uuid REFERENCES users(id), plan text,  -- '1mo'|'2mo'|'3mo'|'6mo'
  started_at timestamptz, expires_at timestamptz, esi_included boolean DEFAULT
true,
  status text )                           -- active | expired | refunded
```

2.5 Practice & mastery (adaptive engine)

```
attempts (
  id uuid PK, user_id uuid, question_id text REFERENCES nodes(id),
  chosen jsonb,               -- index | [indices] | {row:col} | bowtie parts
  correct boolean, ms int, ability_at numeric,   -- ability estimate at time of
answer
  created_at timestamptz )

mastery (
  user_id uuid, node_id text REFERENCES nodes(id),
  score real,                 -- 0..1 estimated mastery
  confidence real,            -- self-rated / derived
  attempts int DEFAULT 0, correct int DEFAULT 0,
  last_seen timestamptz, next_review timestamptz,   -- spaced repetition
  ease real DEFAULT 2.5, interval_days int DEFAULT 0,
  PRIMARY KEY (user_id, node_id) )

streaks ( user_id uuid PK, current int, longest int, last_claim date, points int
```

```
)

question_stats (                -- powers the "% chose X" answer stats + IRT
recalibration
  question_id text PK, exposures int, correct int, option_counts jsonb,
  irt_difficulty real )         -- recalibrated from real data over time
```

2.6 Community (Twitter-style — text feeds + threads)

```
threads ( id uuid PK, author_id uuid, root_post_id uuid, reply_count int,
updated_at )
posts (
  id uuid PK, thread_id uuid, author_id uuid, parent_id uuid,   -- null = root
  kind post_kind, body text NOT NULL,        -- text only; NO video uploads
  like_count int DEFAULT 0, tags text[],     -- optional topic tags -> nodes
  created_at timestamptz )
-- visibility: readable by visitors; posting requires role >= member
-- profiles do NOT render a feed (product decision)
```

---

3. Node body templates (JSONB shapes)

Each `type` validates its `body` against a template (extend the `tools/build_qbank.py` validator pattern to all types). Examples:

DIS (disease)

```
{ "definition":"", "causes":[], "risk_factors":[], "pathophysiology":"",
  "signs_symptoms":[], "assessment":[], "diagnostics":[], "lab_findings":[],
  "nursing_diagnoses":[], "interventions":[], "medications":[],
"patient_teaching":[],
  "complications":[], "priority_actions":[], "nclex_tips":[], "memory_aids":[] }
```

MED (medication)

```
{ "generic":"", "brand":[], "drug_class":"", "pharm_class":"",
"therapeutic_class":"",
  "pronunciation":"", "overview":"", "moa":"", "uses":[], "dosage":
{"adult":"","peds":""},
  "routes":[], "admin_tips":[], "contraindications":[], "warnings":[],
"black_box":"",
  "side_effects":{"common":[],"serious":[],"life_threatening":[]},
  "nursing":{"pre":[],"during":[],"post":[]}, "patient_teaching":[] }
```

LAB

```
{ "abbrev":"", "specimen":"", "normal":{"adult":"","peds":""}, "critical":
{"low":"","high":""},
  "purpose":"", "high_causes":[], "low_causes":[], "nursing_actions":[],
"notify_threshold":"",
  "patient_teaching":[] }
```

PRO (procedure)

```
{ "settings":[], "purpose":"", "indications":[], "equipment":[], "preparation":
[],
  "steps":[], "risks":[], "responsibilities":{"before":[],"during":[],"after":
[]},
  "patient_teaching":[], "documentation":[] }
```

(SKL, ANA, FLS, VID, IMG, EDU, PATH have their own locked shapes — same pattern.)

---

4. Question object (in `nodes.body`) — all item types

Common fields (every question): `qtype · stem · tip · pearl · references[] · learning_objective · bloom · est_time_sec`. Type-specific:

```
// mc
{ "qtype":"mc", "opts":[{"t":"","r":""}×4], "correct":1, "pct":[..] }
// sata
{ "qtype":"sata", "opts":[{"t":"","r":""}≥3], "correct":[0,1,4] }   // all-or-
nothing
// matrix
{ "qtype":"matrix", "cols":["",""], "rows":[{"t":"","correct":0,"r":""}] }
// bowtie
{ "qtype":"bowtie",
  "condition":{"prompt":"","options":[{"t":"","r":""}],"correct":0},
  "actions":{"prompt":"","pick":2,"options":[...],"correct":[0,1]},
  "parameters":{"prompt":"","pick":2,"options":[...],"correct":[0,1]} }
// dropdown (cloze): stem has {{1}} tokens
{ "qtype":"dropdown", "blanks":[{"id":1,"options":
[{"t":"","r":""}],"correct":2}] }
// ordered
{ "qtype":"ordered", "items":[{"t":"","r":""}], "correct_order":[2,0,1,3] }
// hotspot
{ "qtype":"hotspot", "image_id":"IMG-..", "regions":
[{"id":"","correct":true,"r":""}] }
// trend / audio / image : mc/sata payload + a media_id and time-series/asset
```

Rule (validator-enforced): every option/row/blank carries its own rationale (right AND wrong). Difficulty (1–5) required on every question.

---

5. API contract

Base: `/api/v1`. All list endpoints paginate (§0). Content endpoints auto-filter by entitlement; gated items return a `locked:true` stub for non-subscribers.

Auth

```
POST /auth/register        {email,password,name}          -> {user, token}
POST /auth/login           {email,password}               -> {user, token}
GET  /me                   -> {user, entitlement, streak}
PATCH /me                  {exam_date,target,...}
```

Knowledge (NKE)

```
GET /node/:id              -> { node, related:
{diseases[],meds[],labs[],procedures[],

questions[],cases[],videos[],flashcards[]},
                                connections:[{id,title,relation,weight}] }
GET /nodes?type=&system=&specialty=&difficulty=&status=published
GET /search?q=lasix        -> unified: { primary, diseases[], meds[], labs[],
procedures[],
                                      questions[], cases[], flashcards[],
videos[],
                                      esi_hint }
GET /node/:id/why          -> next "Why?" chain steps (linked nodes)
```

Questions & practice

```
GET  /questions?system=&specialty=&type=&difficulty=&client_need=&cj_step=&free=
GET  /question/:id
POST /attempt              {question_id, chosen, ms}
                           -> { correct, correct_answer, rationales, tip, pearl,
                                stats:{option_counts}, mastery_delta,
next_suggested }
GET  /me/next              -> Smart Study: the next best item (adaptive, §6)
GET  /me/session?mode=tutor|timed&n=  -> a generated set
```

Dashboard & mastery

```
GET /me/dashboard          -> { readiness, pass_probability, by_domain:
[{system,score}],
                                weakest[], strongest[], streak, questions_done,
                                ngn_score, med_score, lab_score,
specialty_readiness[] }
GET /me/mastery/:node_id
GET /me/reviews            -> due spaced-repetition items
```

Esi (AI)

```
POST /esi/ask   {message, context?}  -> { answer, cited_node_ids[],
follow_up_practice[],
                                          related[] }   // structured content
first, then plain language
```

Esi reads the user's mastery/history; subscriber-gated; educational-only disclaimer attached.

Community (Twitter-style)

```
GET  /threads?cursor=      -> feed (readable by visitors)
GET  /thread/:id           -> root + replies
POST /threads              {body, tags?}          // role >= member; text only
POST /thread/:id/reply     {body, parent_id?}
POST /post/:id/like
```

No media-upload endpoint (no user video/image posting). Profiles expose learning stats, **not a feed**.

Entitlements / billing (Phase 2 integration)

```
GET  /plans                -> [{id:'1mo',price,anchor,save,esi_included:true},
...]
POST /checkout             {plan}   -> payment session (Stripe later)
POST /webhooks/billing     (server-to-server)   -> writes entitlements
```

---

6. Adaptive engine (attempt → mastery → next)

1. **On `POST /attempt`:** record it; update `mastery(user,node)` — correct raises `score`, sets `ease` / `interval_days` / `next_review` (SM-2-style spaced repetition); update `question_stats` (exposures, option_counts) for answer stats + future IRT recalibration.
2. **Graph propagation:** a miss lowers priority-adjacent nodes via `edges` (weighted) — those become higher priority in `/me/next`. This is the personalized learning path.

3. **CAT selection (`/me/next` in exam mode):** maintain the user's **ability estimate** on the difficulty scale; pick the next unseen item whose `difficulty` / `irt_difficulty` is nearest the estimate; update estimate after each answer; apply the **75–145 item** window and a confidence stop rule for readiness exams.
4. **Readiness / pass probability:** logistic function of the ability estimate vs. the passing standard, shown as Low / Borderline / High / Very High on the dashboard and readiness exams.
5. **Recalibration:** as real `attempts` accumulate, recompute each item's `irt_difficulty` from performance (the "pretest" process) — expert Easy/Moderate/Hard is the seed, data is the truth.

---

7. Content workflow, versioning, QC

- **Status flow:** `draft → in_review → published` (archived to retire). Non-published never served to learners.
- **AI drafts, humans approve:** AI generates in batches; a nurse SME reviews before `published`. Never auto-publish clinical content.
- **Versioning:** bump `nodes.version` on edit; keep an audit trail.
- **QC gate (per item, enforced by validator):** verified answer · rationale on every option · NGN wording · no ambiguity · difficulty tagged · dedup (unique id) · correct placement · references logged · 100% original.
- **Seed pipeline:** today's `data/qbank/*.json` + `tools/build_qbank.py` validator is the prototype of this ingest+validate flow — the web app formalizes it into the DB.

---

8. Non-functional & security

- **Auth:** JWT (short-lived access + refresh); passwords hashed (argon2/bcrypt).
- **Authorization:** entitlement checks on gated content; roles for authoring.
- **Rate limits** on `/esi/ask`, `/attempt`, community POSTs.
- **Caching/CDN** for published nodes; anti-scrape at the CDN (design bible policy) + `robots.txt`.
- **Privacy:** learner data is sensitive; encrypt at rest, least-privilege access, export/delete on request.
- **Observability:** log attempts + esi calls for analytics and recalibration.

---

9. How this maps to what exists

- The **qbank batch pipeline** (`data/qbank/*.json`, `tools/build_qbank.py`, loader) already models §4 (question object), §7 (validation/QC), and free-gating — it's the working seed of this schema.
- The **static site** stays a marketing/prototype front; the **web app** is the DB-backed build against these APIs.

- Nothing here is built yet — this is the contract to approve, then implement foundation-first (DB + auth + core endpoints), then pour content in.

End of Database & API draft v1. On approval: (a) finalize table types with the developer, (b) scaffold the web-app foundation (DB + auth + `/node`, `/search`, `/questions`, `/attempt`, `/me/dashboard`), (c) migrate the existing question batches into the DB as the first real content.